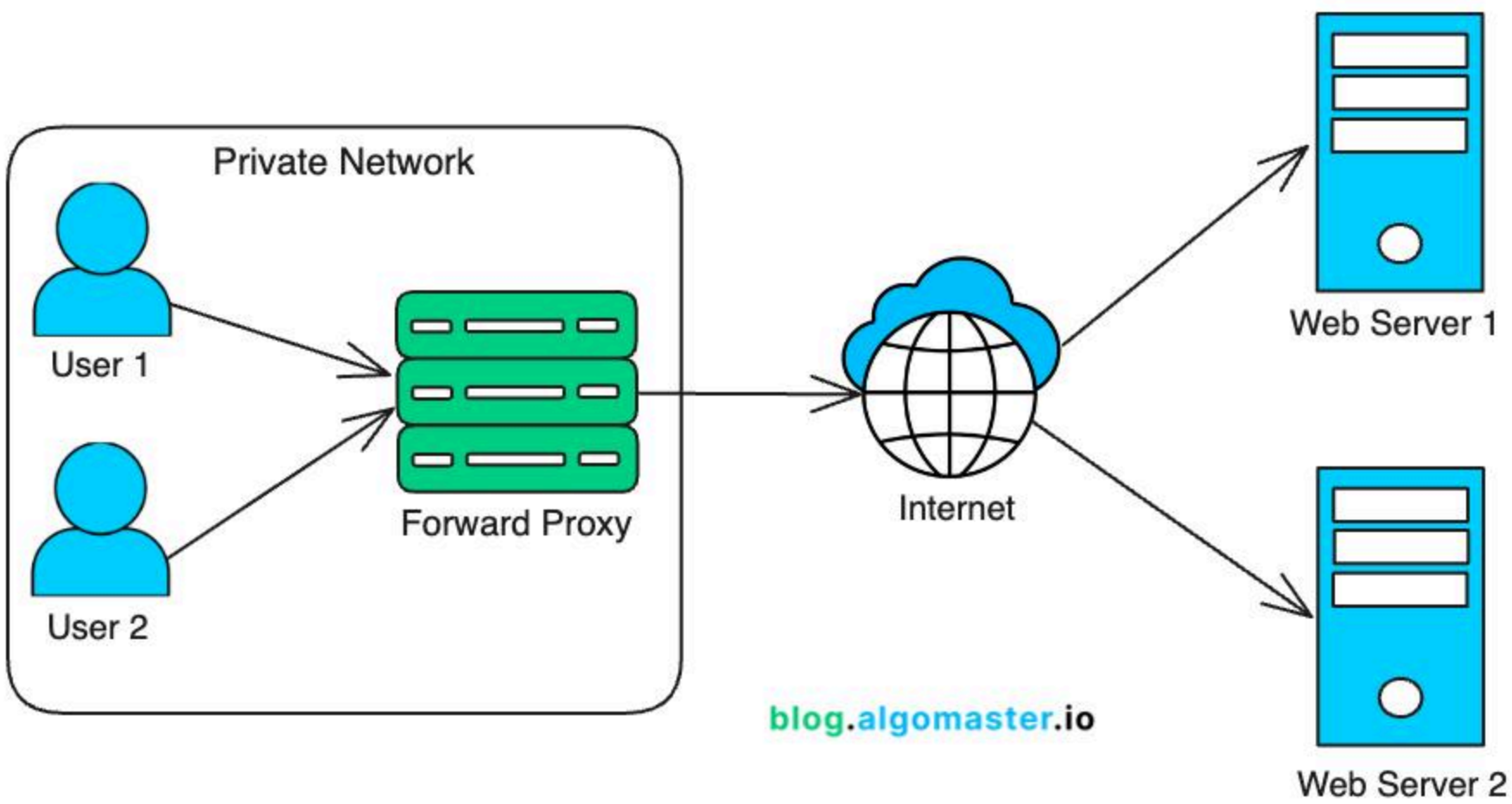
10. Proxy

Proxy

The Core Idea

- A proxy is an intermediary that sits between two parties in a communication
- Instead of client talking directly to server, the proxy sits in the middle
- The proxy receives the request, does something with it, and forwards it
- "Something" could be — hiding identity, filtering, load balancing, caching, security
- There are different types of proxies and the key distinction is — who does the proxy represent and where does it sit

Forward Proxy



What it is

- A forward proxy sits between the client and the internet
- It represents the client — it acts on behalf of the client
- The client sends its request to the forward proxy instead of directly to the server
- The forward proxy forwards the request to the actual server
- The server sees the proxy's IP, not the client's IP
- The client knows it is using a proxy
- The server does not know the real client exists — it only sees the proxy

The flow

- Client wants to reach google.com
- Instead of connecting directly, client sends request to Forward Proxy
- Forward Proxy receives the request, forwards it to google.com using its own IP
- google.com responds to the Forward Proxy
- Forward Proxy returns the response to the client
- google.com never knew the original client's IP

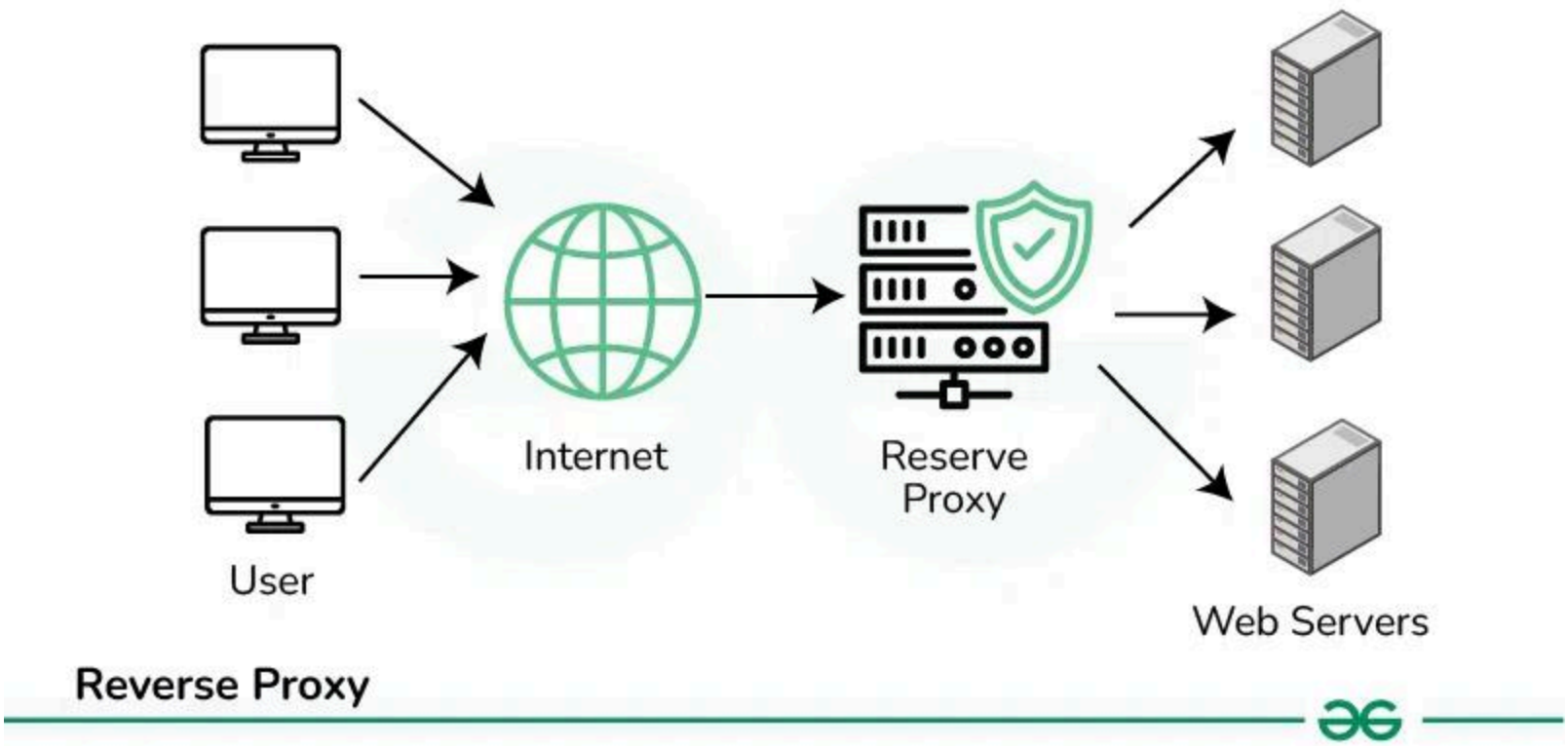
What it does

- Hides the client's identity from the server
- Filters outbound traffic — blocks certain websites or content categories
- Caches responses — if 100 employees all request the same page, proxy fetches it once and serves from cache
- Bypasses geo-restrictions — proxy sits in a different country, requests appear to come from that country
- Logs and monitors outbound traffic — useful for compliance and security auditing
- Enforces access control — only allow certain users to access certain sites

Use cases

- Corporate networks
  - Company deploys a forward proxy for all employee internet traffic
  - Blocks access to social media, gambling, or malicious sites
  - Logs all outbound requests for compliance and security auditing
  - All traffic goes through one point — easy to monitor and control
- Privacy and anonymity
  - VPNs work similarly to forward proxies at a conceptual level
  - User's real IP is hidden from websites they visit
  - Used by journalists, activists, or anyone needing anonymity online
- Bypassing geo-restrictions
  - A user in a country where a service is blocked routes traffic through a proxy in another country
  - The service sees the proxy's country IP and allows access
- Content filtering for schools or ISPs
  - Schools deploy forward proxies to prevent students from accessing inappropriate content
  - ISPs in some countries use forward proxies to enforce government content restrictions
- Caching in large organisations
  - If 500 employees all open the same news site in the morning
  - Forward proxy fetches it once, caches it, serves all 500 from cache
  - Saves bandwidth significantly

Reverse Proxy



What it is

- A reverse proxy sits between the internet and your backend servers
- It represents the server — it acts on behalf of the server
- The client sends its request to the reverse proxy thinking it is the actual server
- The reverse proxy forwards the request to one of the backend servers
- The client does not know the real backend servers exist — it only sees the reverse proxy
- The client does not know it is talking to a proxy at all

The flow

- Client wants to reach api.example.com
- DNS points api.example.com to the reverse proxy's IP
- Client sends request to the reverse proxy
- Reverse proxy decides which backend server to forward to
- Backend server processes and responds to the reverse proxy
- Reverse proxy returns the response to the client
- Client thinks it talked directly to api.example.com — it has no idea backend servers exist

What it does

- Load balancing — distributes incoming requests across multiple backend servers
- SSL termination — handles the TLS handshake so backend servers don't have to
- Caching — caches responses from backend servers, serves repeated requests without hitting the backend
- Compression — compresses responses before sending to clients, saves bandwidth
- Security — hides backend server IPs, absorbs DDoS traffic, enforces rate limiting
- Authentication — can validate tokens before requests reach the backend
- Routing — routes different URLs to different backend services (critical in microservices)

Use cases

- Load balancing
  - You have 5 application servers running your API
  - Reverse proxy distributes traffic evenly across all 5
  - If one server goes down, reverse proxy detects it via health checks and stops sending traffic to it
  - Clients never know one server is down
- SSL termination
  - TLS handshake is computationally expensive
  - Instead of every backend server handling TLS, the reverse proxy handles it once
  - Communication between reverse proxy and backend servers happens over plain HTTP inside the private network
  - Simplifies certificate management — only the reverse proxy needs the SSL certificate
  - Backend servers are simplified and faster
- Hiding backend infrastructure
  - Your actual servers' IPs are never exposed to the internet
  - Attackers cannot directly target your application servers
  - All traffic must go through the reverse proxy which can filter, rate limit, and block malicious traffic
- Microservices routing
  - You have separate services: auth service, product service, order service
  - All exposed under one domain: api.example.com
  - Reverse proxy routes:
    - api.example.com/auth → auth service on port 3001
    - api.example.com/products → product service on port 3002
    - api.example.com/orders → order service on port 3003
  - Client sees one clean API, reverse proxy handles the routing internally
- Caching static content
  - Reverse proxy caches images, JS files, CSS — static assets
  - Subsequent requests for the same asset are served directly from the proxy's cache
  - Backend server is never hit — massive reduction in load
- Rate limiting and DDoS protection
  - Reverse proxy tracks request rates per IP
  - If an IP sends 1000 requests per second, reverse proxy blocks it
  - Backend servers are shielded from abusive traffic

Nginx as a Reverse Proxy

- Nginx is one of the most widely used reverse proxies in production systems
- Originally built as a web server but became the standard reverse proxy for most architectures
- Used by companies like Netflix, Dropbox, WordPress

What Nginx does as a reverse proxy

- Receives incoming HTTP/HTTPS requests
- Terminates TLS — handles the SSL certificate, decrypts the request
- Routes the request to the appropriate backend server based on configuration rules
- Load balances across multiple backend instances
- Caches responses for configured routes

- Compresses responses using gzip before sending to clients
- Serves static files directly without touching the backend at all
- Adds or strips headers before forwarding requests

How Nginx routing works conceptually

- You define server blocks and location blocks in the Nginx config
- A server block defines which domain and port Nginx listens on
- A location block defines what to do with requests matching a specific path
- Example logic:
  - Requests to /api → forward to backend app server on port 3000
  - Requests to /static → serve directly from the filesystem
  - Requests to /images → serve from cache or forward to image service
- This routing logic is what makes Nginx the front door of almost every production backend

Why Nginx specifically

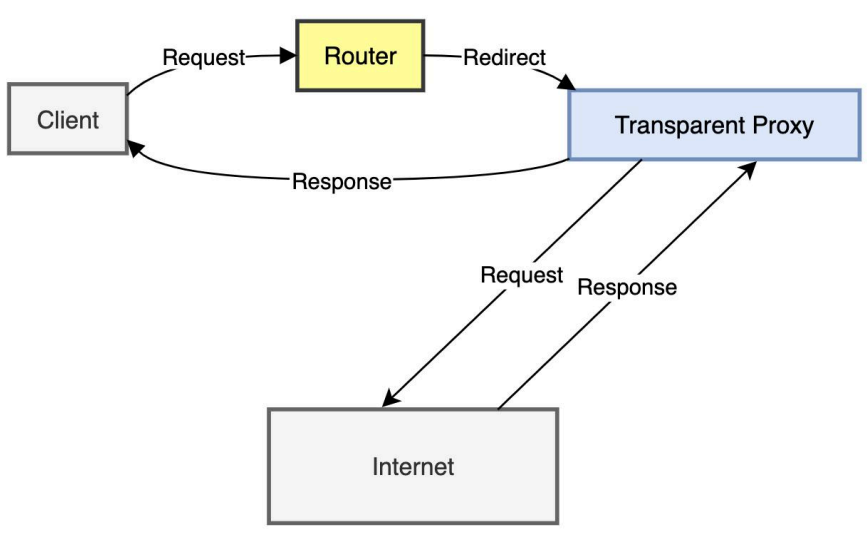
- Asynchronous, event-driven architecture — handles thousands of concurrent connections with very low memory
- Unlike older servers like Apache which used one thread per connection, Nginx uses a small number of worker processes each handling thousands of connections
- This makes it extremely efficient under high traffic

Forward vs Reverse Proxy Comparison

- Who it represents
  - Forward Proxy → represents the client, acts on behalf of clients
  - Reverse Proxy → represents the server, acts on behalf of servers
- Who knows about the proxy
  - Forward Proxy → client knows about it, server does not
  - Reverse Proxy → server knows about it, client does not
- What IP is hidden
  - Forward Proxy → hides the client's IP from the server
  - Reverse Proxy → hides the server's IP from the client
- Where it sits
  - Forward Proxy → sits at the edge of the client's network, between clients and the internet
  - Reverse Proxy → sits at the edge of the server's network, between the internet and backend servers
- Primary use cases
  - Forward Proxy → privacy, content filtering, bypassing restrictions, corporate monitoring
  - Reverse Proxy → load balancing, SSL termination, caching, security, microservices routing
- Who configures it
  - Forward Proxy → configured by the client or the client's network admin
  - Reverse Proxy → configured by the server owner or infrastructure team

---

Transparent Proxy



What it is

- A transparent proxy intercepts traffic without the client knowing it exists
- The client does not configure anything — it thinks it is talking directly to the server
- The network itself redirects traffic through the proxy transparently
- Also called an intercepting proxy or inline proxy

How it works

- Traffic flows from client toward the internet as normal
- A network device (router, firewall) intercepts the traffic and redirects it to the proxy
- The proxy processes the request and forwards it to the destination
- Client never knew a proxy was involved

Use cases

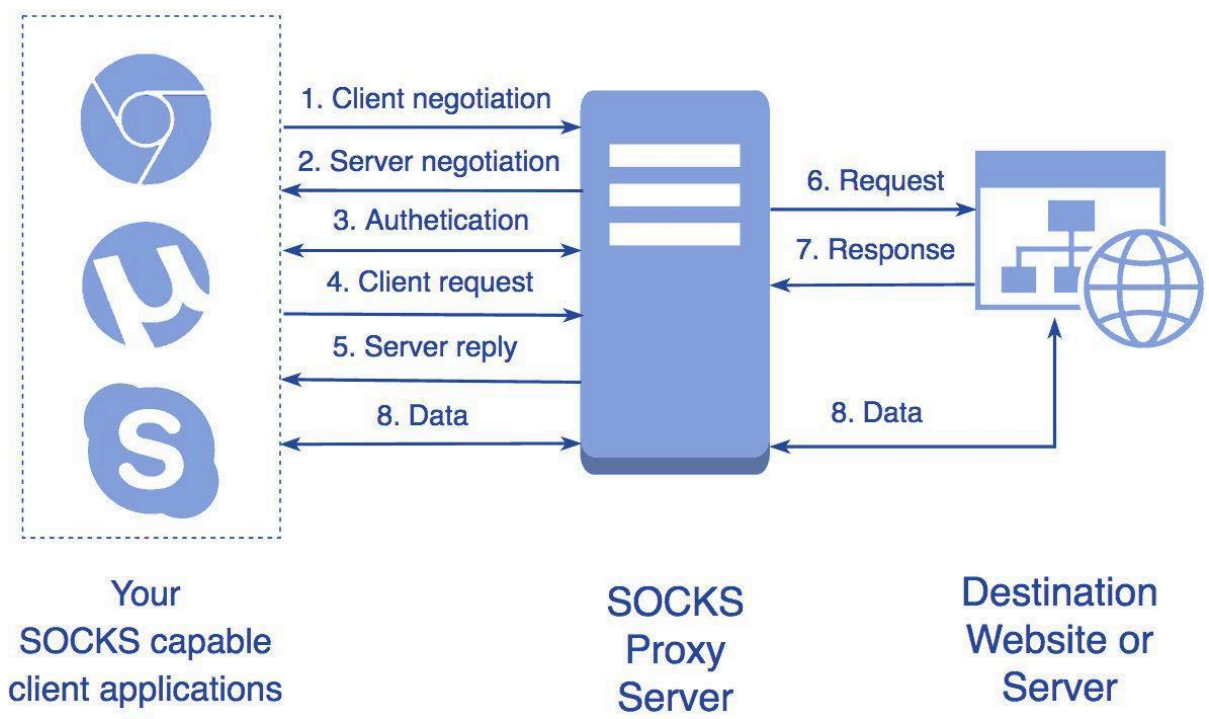
- ISP-level caching — ISP transparently caches popular content to save bandwidth on their network
- Parental controls at the router level — home router silently filters traffic through a proxy
- Corporate networks — employees do not need to configure anything, proxy is enforced at the network level
- Content filtering by governments or ISPs — traffic is silently inspected and filtered
- CDN edge nodes — in some configurations, CDNs act as transparent proxies for cached content

Key characteristic

- Because the client does not know about the proxy, it does not send a Proxy-Authorization header
- The proxy must work purely by intercepting and inspecting the traffic at the network level
- HTTPS traffic is difficult for transparent proxies to inspect because it is encrypted
  - Some corporate proxies do SSL inspection — they act as a man-in-the-middle, terminate the TLS connection, inspect the traffic, and re-encrypt it
  - This is why corporate laptops often have a company root certificate installed — it allows the proxy to present its own certificates trusted by the browser

---

SOCKS Proxy



What it is

- SOCKS stands for Socket Secure
- A SOCKS proxy operates at a lower level than HTTP proxies
- HTTP proxies understand HTTP — they read and modify HTTP requests
- SOCKS proxy does not understand the application protocol at all
- It just forwards raw network packets from client to destination
- It is protocol-agnostic — works with HTTP, HTTPS, FTP, SMTP, torrent traffic, anything

How it works

- Client connects to the SOCKS proxy server
- Client tells the proxy "connect me to destination IP and port X"
- SOCKS proxy opens a TCP connection to that destination

- All data flows through the SOCKS proxy as raw bytes — proxy does not read or modify it
- Proxy just shuttles bytes back and forth

SOCKS4 vs SOCKS5

- SOCKS4 — supports only TCP, no authentication
- SOCKS5 — supports TCP and UDP, supports authentication (username/password), supports DNS resolution through the proxy
  - DNS through the proxy is important — without it, DNS queries go directly from the client, leaking which domains you are visiting even if traffic goes through the proxy

Use cases

- Torrenting and P2P traffic — HTTP proxies cannot handle these protocols, SOCKS can
- Bypassing firewalls — since SOCKS is protocol-agnostic, it can tunnel any type of traffic through a firewall that only allows certain ports
- Privacy — hides client IP at the TCP level across any protocol
- SSH tunnelling — SSH can act as a SOCKS5 proxy, tunnelling all traffic through an encrypted SSH connection to a remote server
- Gaming — bypass IP bans or geo-restrictions on game servers

SOCKS vs HTTP proxy

- HTTP proxy understands HTTP — can cache, modify headers, filter content, inspect requests
- SOCKS proxy understands nothing about the application layer — just moves bytes, no inspection, no modification
- HTTP proxy is more feature-rich for web traffic specifically
- SOCKS proxy is more versatile because it works with any protocol
- SOCKS proxy is faster for raw throughput because it does no application-level processing

---

Summary

- Forward proxy — you hire someone to go to the store for you, the store does not know who really wants the product
- Reverse proxy — a company's receptionist who takes all calls and routes them to the right department, callers do not know which department they are talking to
- Transparent proxy — a security guard who quietly checks your bag as you walk through without stopping you
- SOCKS proxy — a dumb pipe that carries any kind of package between two points without knowing or caring what is inside